

Project Status Report: Brag Board

Project Name: Brag Board: Internal Employee Recognition Wall

Name: Gauresh Nandkumar Govekar

GitHub: <https://github.com/gaureshconnects/Bragboard.git>

Email: gauresh.clg@gmail.com

Summary:

This report summarizes the completion of Task 3: **Create a Shout-out Form with Recipient Tagging**, which involves enabling users to write shout-outs, tag recipients, and store the posts in the database with tagged users. This task builds upon the Week 2 dashboard and department-scoping foundation. The implementation ensures secure and structured storage of posts and tagged users while maintaining efficient retrieval for frontend display.

Completed Tasks for Week 2. Task: User Dashboard and Department-wise Access

Frontend Implementation:

Shout-out Form UI:

- Developed a responsive shout-out form using React and Tailwind CSS.
- The form includes a text area for post content and a multi-select dropdown for tagging colleagues within the same department.
- The dropdown dynamically fetches users from the backend filtered by department, ensuring only eligible recipients can be tagged.

Form Validation:

- Added client-side validation to ensure that the post text is not empty.
- Users can tag multiple colleagues, and duplicate selections are prevented.

Submission & Feedback:

- On form submission, the shout-out and tagged users are sent to the backend via an Axios POST request.
- Success and error notifications are displayed to the user using alert/toast components.

Responsive Design:

- The form layout and dropdown are fully responsive and optimized for desktop, tablet, and mobile devices using Tailwind flex and grid utilities.

Backend Implementation:

API Endpoint for Shout-out Creation:

- Implemented the `/posts/create` endpoint in Fast API.
- Accepts the post content, `author_id`, and a list of `tagged_user_ids`.
- Stores the post in the posts table and adds tagged users in the `post_tags` many-to-many association table.

Database Integration:

- Post and User models updated to include many-to-many relationships for tagged users.
- Eager loading (selectin load) is used for retrieving posts with author and tagged user details efficiently in async queries.

Validation & Security:

- Author ID is validated to ensure the user is authenticated.
- Tagged users are validated against department scope to prevent tagging users from unauthorized departments.
- The endpoint uses JWT authentication to ensure only authorized users can create posts.

Retrieval of Posts with Tagged Users:

- Created `/posts/all` endpoint to fetch all posts along with author names and tagged users' names.
- Ensured async-safe eager loading to avoid SQL Alchemy lazy-loading issues in async context.

Expected Outcomes:

- Users can create shout-outs and tag colleagues securely.
- Tagged users are stored in the database and correctly linked to the posts.
- Posts can be retrieved with author and tagged user names for display on the dashboard or team wall.
- Backend ensures department-scoped tagging and access control.
- Forms are fully responsive and user-friendly.